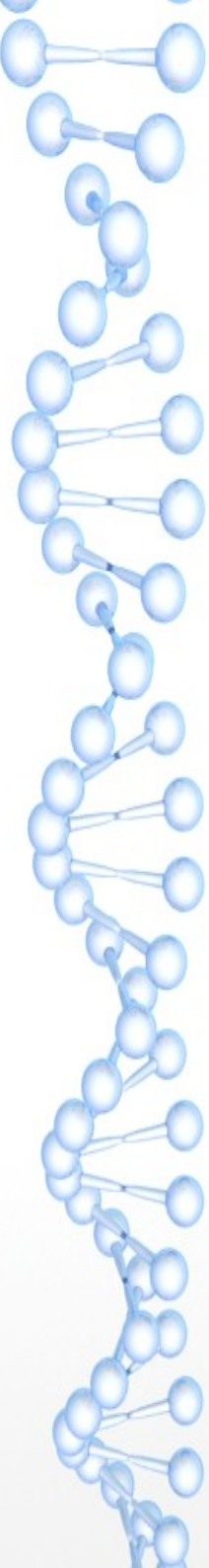


IST602: Web Technologies and Standards

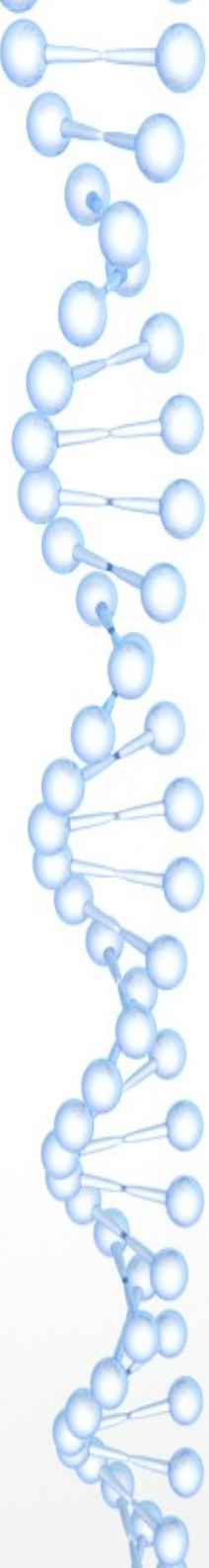
Introduction to TypeScript

Denis L. Nkweteyim



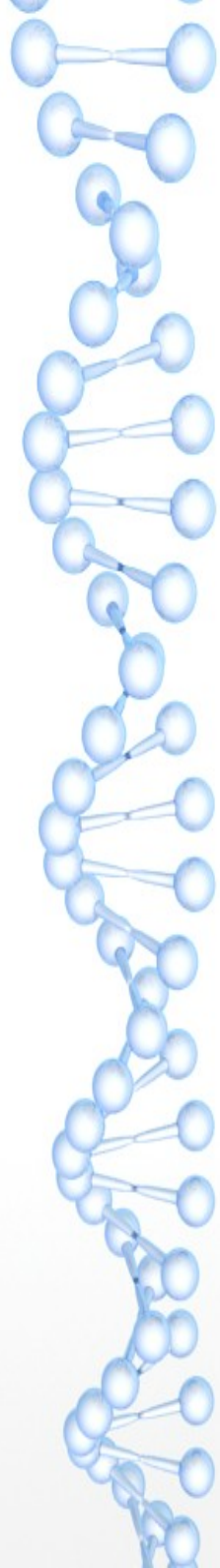
Outline

- **Intro to TypeScript**
- TypeScript Data Types
- Function
- Casting
- Class
- Generics



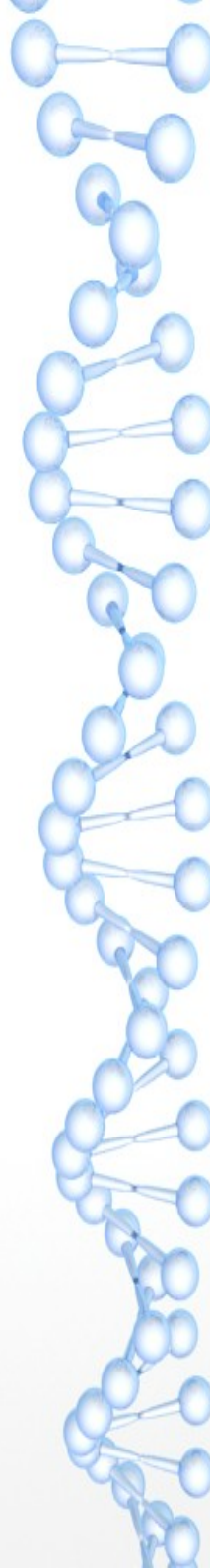
Introduction to TypeScript

- Big idea
 - JavaScript not originally designed for large scale applications
 - Mostly a scripting language
 - Originally lacked structuring mechanisms like Class, Module, Interface



Introduction to TypeScript

- Big idea
 - TypeScript is
 - Compiler for JavaScript code that scales
 - A typed superset of JavaScript that compiles to plain JavaScript
 - Any browser
 - Any host
 - Any OS
 - Open source
 - A Microsoft technology



TypeScript Installation

- Node.js and npm must be installed first, if not already installed in your system
 - Node.js
 - A JavaScript runtime built on Chrome's V8 JavaScript engine
 - Uses an event-driven, non-blocking I/O model that makes it lightweight and efficient
 - An algorithm is called non-blocking if failure or suspension of any thread (e.g., while reading a file from a remote server) cannot cause failure or suspension of another thread
 - npm
 - A package manager for JavaScript and the world's largest software registry
 - Helps you discover packages of reusable code — and assemble them in powerful new ways
 - Package manager for Node.js
- Check the Web for information on how to install Node.js and npm on your system
 - Usually, npm is installed as well when you install Node.js
- TypeScript installation
 - `npm install -g typescript`
 - Note
 - The TypeScript compiler is implemented in TypeScript and can be used in any JavaScript host



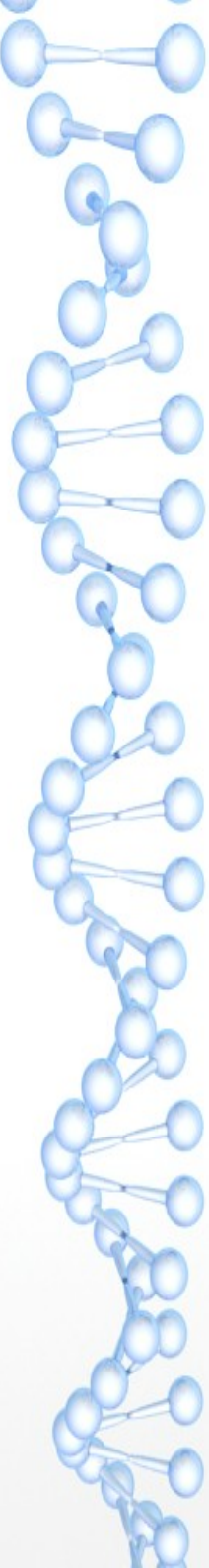
Hello World TypeScript

- Enter the code (greeter.ts) below, left
 - Notice that this TypeScript file is pure JavaScript
 - Which is OK since JavaScript is a subset of TypeScript
- Compile the .ts file
 - tsc greeter.ts
 - A JavaScript file called greeter.js is generated with the same content (below, right)
 - Note: If your version of JavaScript does not support **let**, then **let** will be translated to **var**
- Transpiler
 - Also known as source-to-source (S2S) compiler
 - Transpilers read source code written in one programming language, and produce the equivalent code in another language
 - TypeScript code is transpiled to JavaScript code, which a browser can now interpret

```
function greeter(person) {  
    return "Hello, " + person;  
}  
let user = "Jane User";  
document.body.innerHTML = greeter(user);
```

prim: C

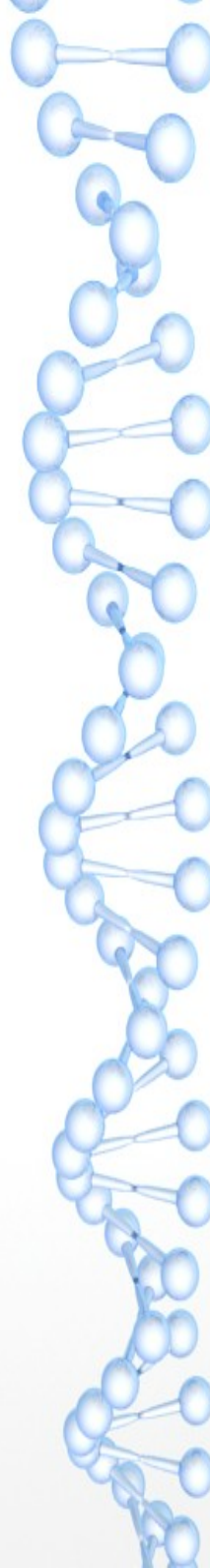
```
function greeter(person) {  
    return "Hello, " + person;  
}  
var user = "Jane User";  
document.body.innerHTML = greeter(user);
```



Modify Code

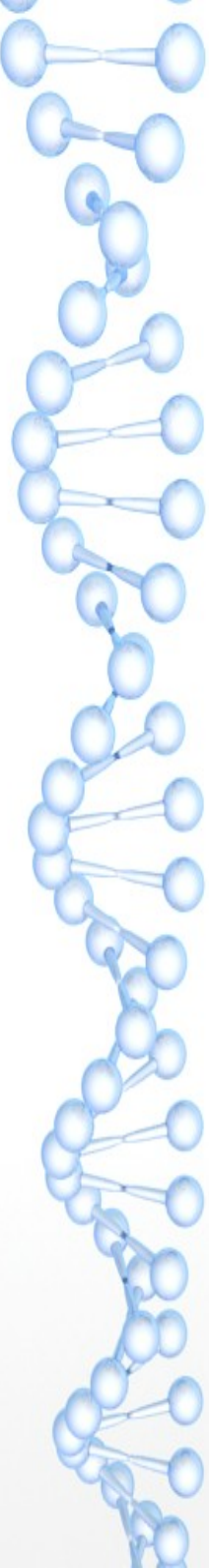
- Add a `: string` type annotation to the 'person' function argument
 - Type annotations in TypeScript are lightweight ways to record the intended contract of the function or variable
 - In this case, we intend the greeter function to be called with a single string parameter
 - The code compiles to the same .js file we had before
 - We can try changing the call to greeter to pass an array instead
 - `let user = [0, 1, 2]`, or
 - Remove the argument to the greeter call
 - `document.body.innerHTML = greeter(user);`
 - In either case, a compilation error is generated

```
function greeter(person: string) {  
    return "Hello, " + person;  
}  
let user = "Jane User";  
document.body.innerHTML = greeter(user);
```



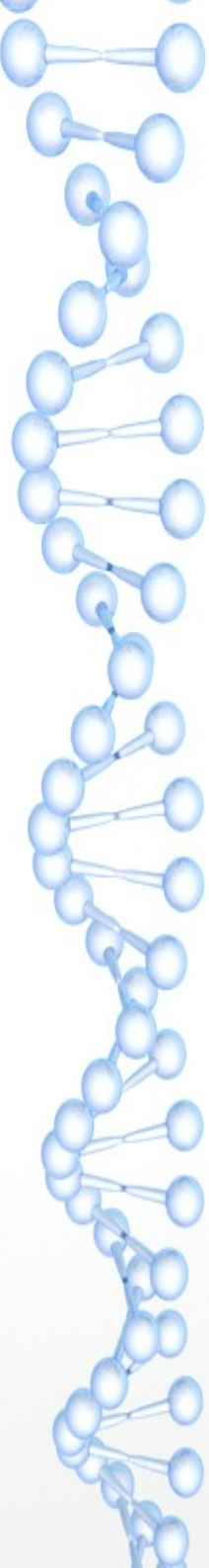
Use the External .js File

```
<!DOCTYPE html>
<html>
  <head><title>TypeScript Greeter</title></head>
  <body>
    <script src="greeter.js"></script>
  </body>
</html>
```



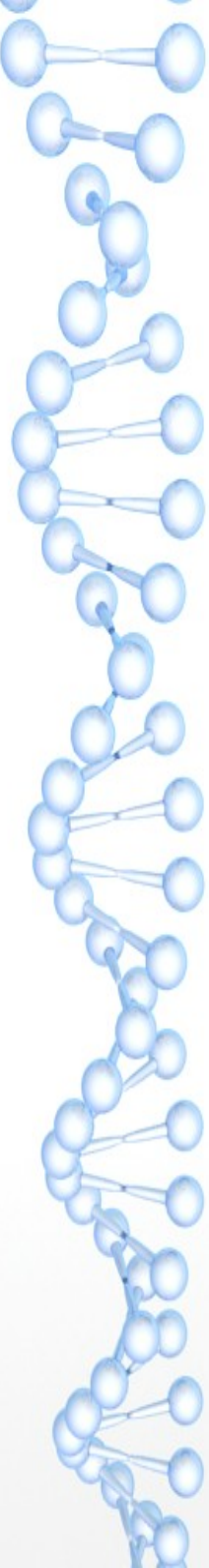
Outline

- Intro to TypeScript
- **TypeScript Data Types**
- Function
- Casting
- Class
- Generics



Typescript data types

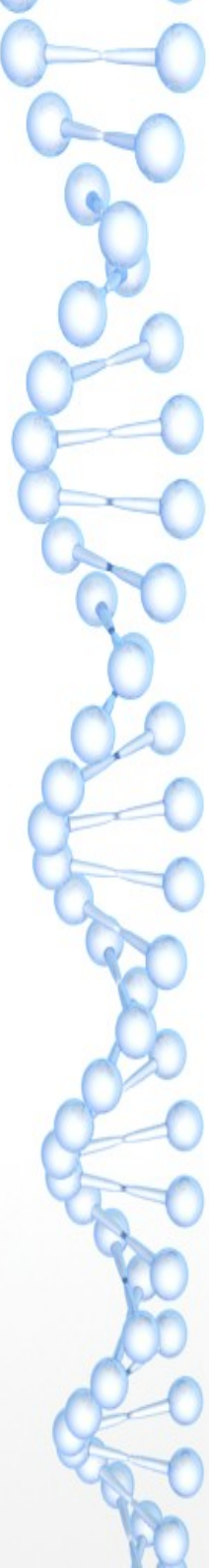
- Any
- Primitive
 - Number
 - Boolean
 - String
 - Void
 - Null - Undefined - Same as JS
- Array
- Enum



Data types - Any

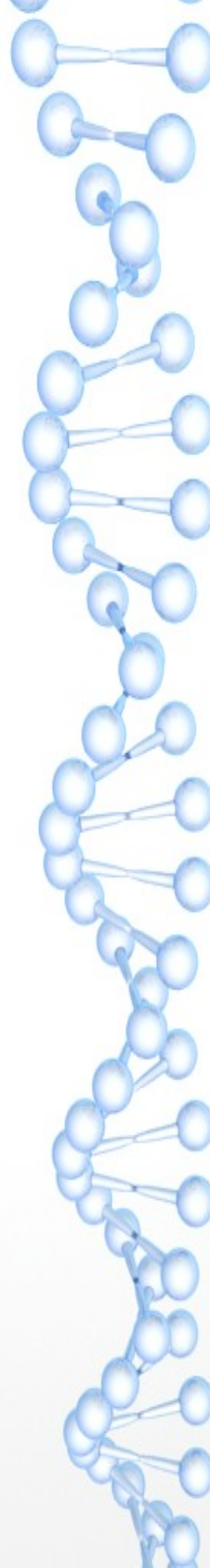
- Super set of all types
 - Used when it is impossible to determine the type
 - Example shows that it is okay to assign values of different types to variable whose type is any

```
var notSure: any = 4;  
notSure = "ABC"  
notSure = 5;
```



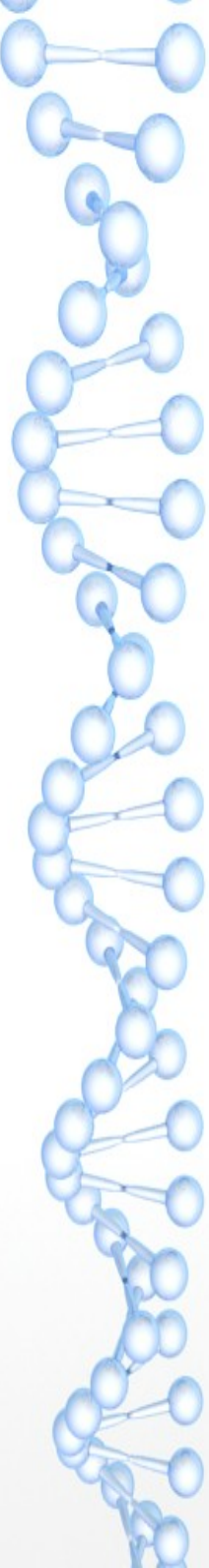
Primitive data types

- number
 - No separate integers and float/double type
 - let decimal: number = 6;
 - let hex: number = 0xf00d;
 - let binary: number = 0b1010;
 - let octal: number = 0o744;
- string
 - Both single quote or double quotes can be used
 - Let color: string = "blue";
 - color = 'red';
 - Note: No separate char type
- boolean
 - true/false value
 - Example: var isOpen = true;
let isDone: boolean = false;
- Void
 - Used in function type returning nothing
- null and undefined
 - function as usual
- Object types
 - class, interface, module



String

- You can also use template strings, which can span multiple lines and have embedded expressions
- These strings are surrounded by the backtick/backquote (```) character, and embedded expressions are of the form `${ expr }`.
 - `let fullName: string = `Bob Bobbington`;`
 - `let age: number = 37;`
 - `let sentence: string = `Hello, my name is ${ fullName }.
I'll be ${ age + 1 } years old next month.`;`
- This is equivalent to declaring sentence like so:
 - `let sentence: string = "Hello, my name is " + fullName + ".\n" + "I'll be " + (age + 1) + " years old next month.";`

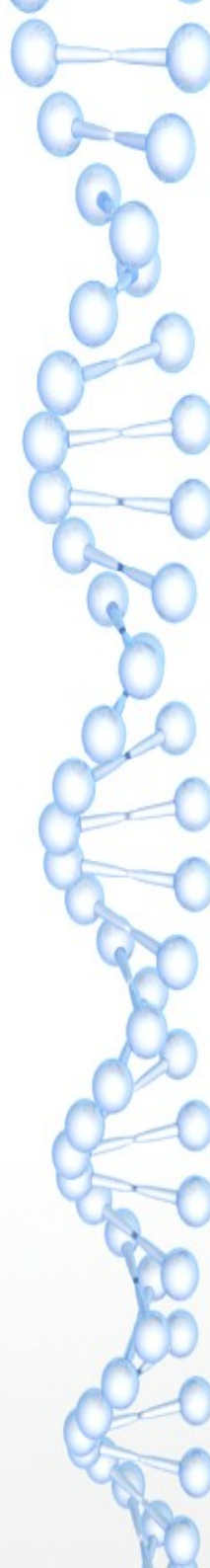


Arrays

```
var cities: string[] = ["Buea", "Limbe", "Kumba"];  
var bools: boolean = [true, false, false];  
var list: any[] = [1, true, "free"]
```

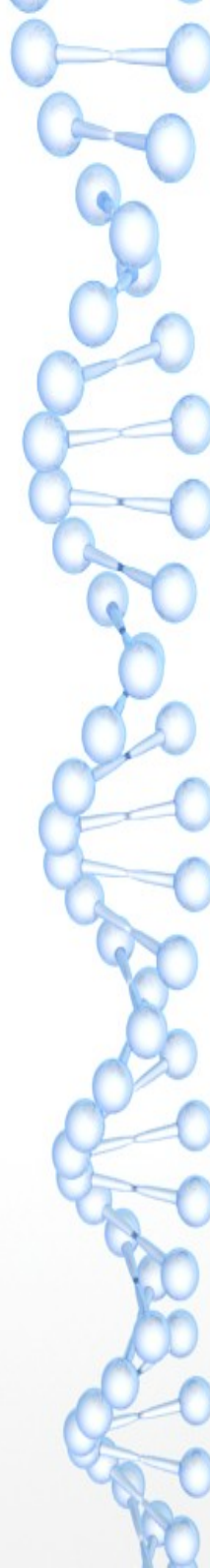
- Alternatively, use generic type

```
var list: Array<number> = [1, 2, 3];  
var countries: Array<string> = ["Cameroon", "Congo"];
```



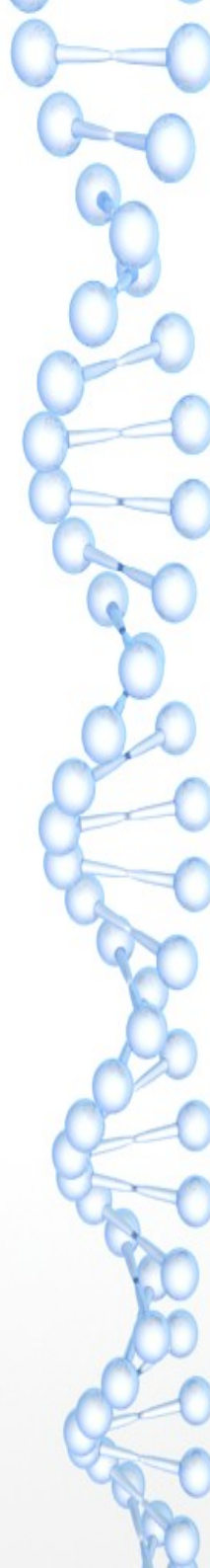
Enum

- A way of giving more friendly names to sets of numeric values
 - `enum Color {Red, Green, Blue}`
 - `let c: Color = Color.Green;`
- By default, enums begin numbering their members starting at 0. But you can change this by manually setting a new value
 - `enum Color {Red = 1, Green, Blue}`
 - `let c: Color = Color.Green; //Green now has the value 2, and Blue the value 3`
- Or, even manually set all the values in the enum:
 - `enum Color {Red = 1, Green = 2, Blue = 4}`
- A handy feature of enums is that you can also go from a numeric value to the name of that value in the enum
 - `enum Color {Red = 1, Green, Blue}`
 - `let colorName: string = Color[2];`
 - `alert(colorName); // Displays 'Green' as it's value is 2 above`



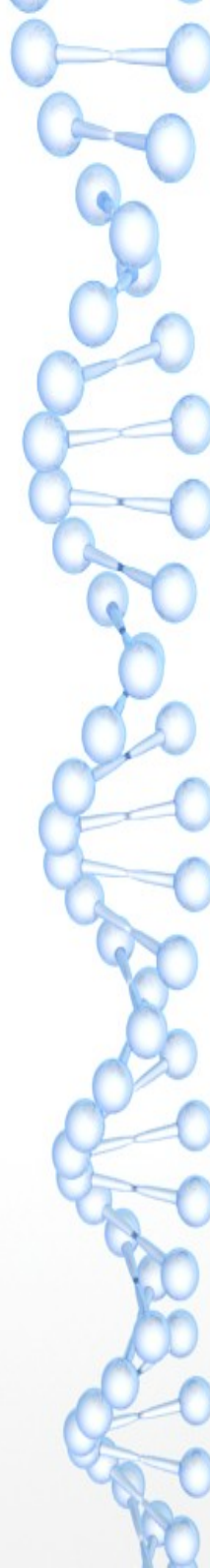
Tuple

- Allows you to express an array where the type of a fixed number of elements is known, but need not be the same. For example, you may want to represent a value as a pair of a string and a number:
 - `// Declare a tuple type`
 - `let x: [string, number];`
 - `// Initialize it incorrectly`
 - `x = [10, "hello"]; // Error`
 - `// Initialize it correctly`
 - `x = ["hello", 10]; // OK`
- When accessing an element with a known index, the correct type is retrieved:
 - `console.log(x[0].substr(1)); // OK`
 - `console.log(x[1].substr(1)); // Error, 'number' does not have 'substr'`
- When accessing an element outside the set of known indices, a union type is used instead
 - `x[3] = "world"; // OK, 'string' can be assigned to 'string | number'`
 - `console.log(x[5].toString()); // OK, 'string' and 'number' both have 'toString'`
 - `x[6] = true; // Error, 'boolean' isn't 'string | number'`



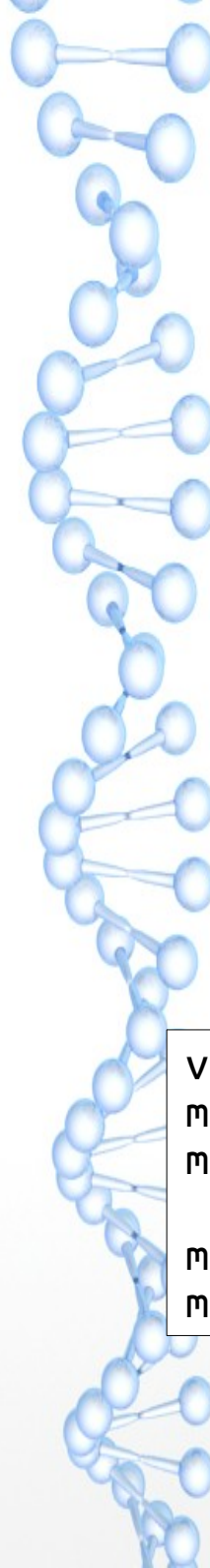
Void

- The absence of having any type at all
- Commonly used as the return type of functions that do not return a value
 - ```
function warnUser(): void {
 • alert("This is my warning message");
}
```
- Declaring variables of type void is not useful because you can only assign undefined or null to them
  - ```
let unusable: void = undefined;
```



Null and Undefined

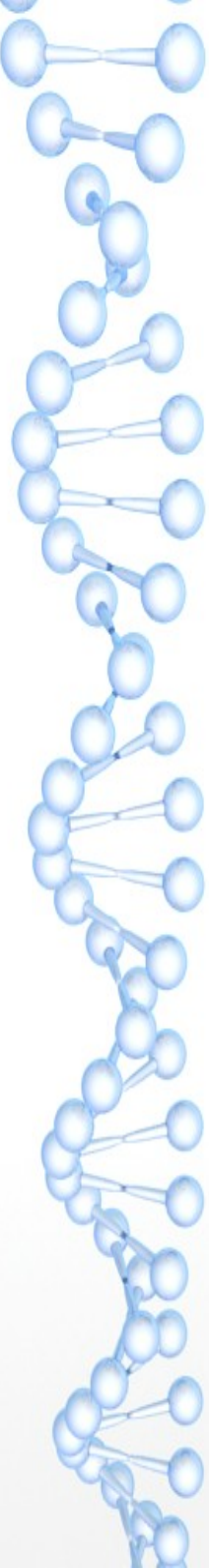
- The values have their own types named null and undefined respectively
- Much like void, they're not extremely useful on their own
 - // Not much else we can assign to these variables!
 - let u: undefined = undefined;
 - let n: null = null;
- By default null and undefined are subtypes of all other types
 - That means you can assign null and undefined to something like number
 - However, when using the --strictNullChecks flag, null and undefined are only assignable to void and their respective types. This helps avoid many common errors



Type Inference

- TypeScript tries to infer types
 - Four ways to variable declaration
 - Type and Value in one statement
 - Type but no Value then Value will be undefined
 - Value but no Type then it will be of Any type but maybe be inferred based on its value
 - Neither Value nor Type then Type will be Any and Value will be undefined
- Example of type inference

```
var message1, message2:string;  
message1 = "Hello world";  
message1 = 4;  
  
message2 = "Hello world";  
message2 = 4; //Error - Type 'number' is not assignable to type 'string'.
```

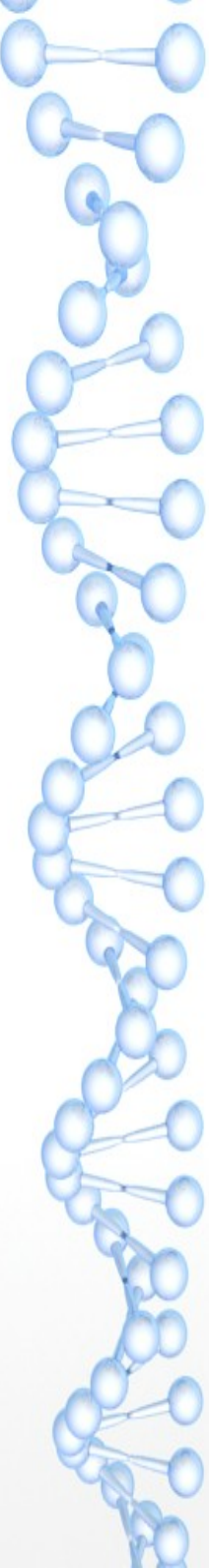


Object Types

- Syntax

```
const car: { type: string, model: string, year:
number } = {
  type: "Toyota",
  model: "Corolla",
  year: 2009
};
```

- If the types are not specified, they are inferred when data is assigned to object properties
- Compare with JavaScript object syntax
 - `const car = {type:"Fiat", model:"500", color:"white"};`



Optional Properties

- They don't have to be defined in the object definition

```
// Error: Property 'mileage' is missing in type  
const car: { type: string, mileage: number } = {  
  type: "Toyota",  
};  
car.mileage = 2000;
```

```
// No error, mileage is optional  
const car: { type: string, mileage?: number } = {  
  type: "Toyota"  
};  
car.mileage = 2000;
```

Type Aliases

- Allow defining types with a custom name, i.e., an Alias
 - Type aliases can be used for primitives like string as well as more complex types such as objects and arrays

```
// Create a new Car using the alias provided
type CarYear = number;
type CarType = string;
type CarModel = string;
type Car = {
  year: CarYear,
  type: CarType,
  model: CarModel
};
```

```
const carYear: CarYear = 2001
const carType: CarType = "Toyota"
const carModel: CarModel = "Corolla"
const car: Car = {
  year: carYear,
  type: carType,
  model: carModel
};
```

Interfaces

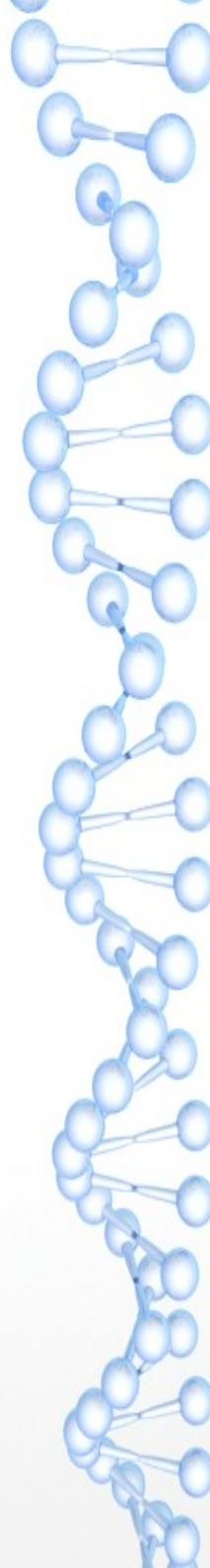
- Interfaces are similar to type aliases, except they only apply to object types

```
interface Rectangle {  
  height: number,  
  width: number  
};  
  
const rectangle: Rectangle = {  
  height: 20,  
  width: 10  
};
```

Extending Interfaces

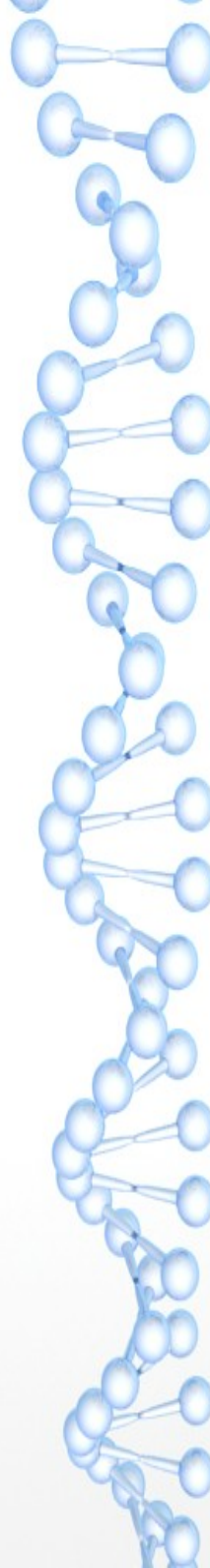
- Extending an interface means you are creating a new interface with the same properties as the original, plus something new

```
interface Rectangle {  
  height: number,  
  width: number  
}  
  
interface ColoredRectangle extends Rectangle {  
  color: string  
}  
  
const coloredRectangle: ColoredRectangle = {  
  height: 20,  
  width: 10,  
  color: "red"  
};
```



Interfaces

- In TypeScript, type-checking focuses on the shape that values have
 - Interfaces fill the role of naming these types, and are a powerful way of defining contracts within your code as well as contracts with code outside of your project
 - Note
 - The TypeScript compiler does not convert interface to JavaScript
 - It uses interface for type checking



Interfaces

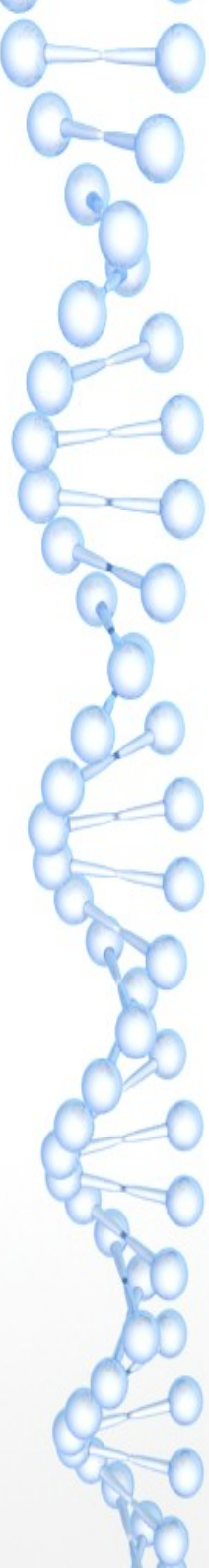
- We now use an interface that describes objects that have a firstName and lastName field
- In TypeScript, two types are compatible if their internal structure is compatible
- This allows us to implement an interface just by having the shape the interface requires, without an explicit implements clause

greeter.ts

```
interface Person {  
    firstName: string;  
    lastName: string;  
}  
function greeter(person: Person) {  
    return "Hello, " + person.firstName + " " + person.lastName;  
}  
let user = { firstName: "Jane", lastName: "User" };  
document.body.innerHTML = greeter(user);
```

greeter.js

```
function greeter(person) {  
    return "Hello, " + person.firstName + " " + person.lastName;  
}  
var user = { firstName: "Jane", lastName: "User" };  
document.body.innerHTML = greeter(user);
```

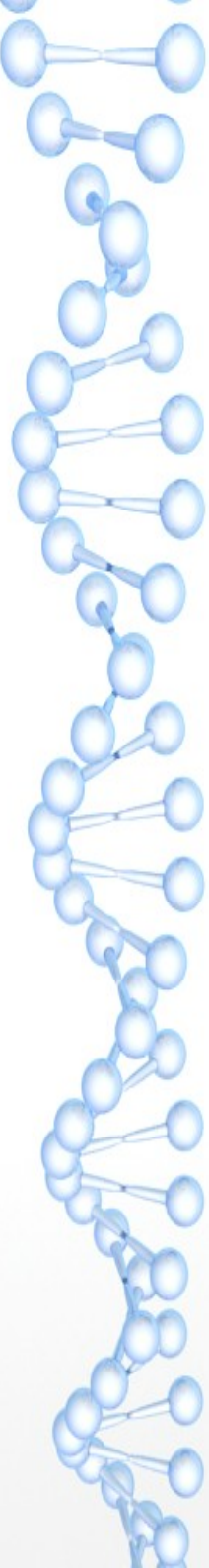


Union Type (|)

- Used when a value can be more than a single type, such as when a property could be string or number

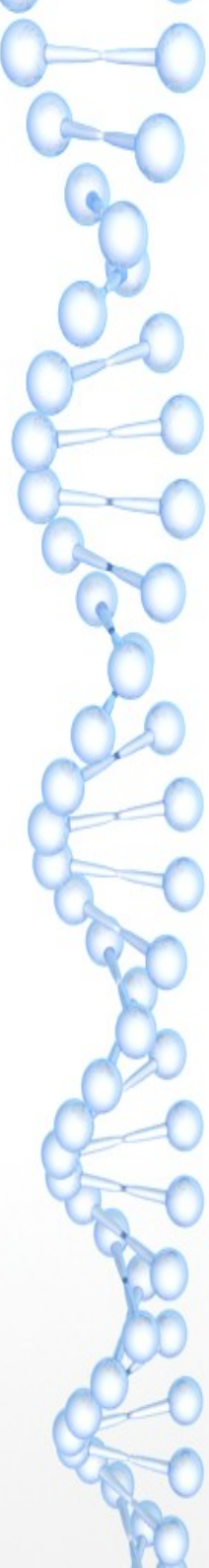
```
function printStatusCode(code: string | number) {  
  console.log(`My status code is ${code}.`)  
}  
printStatusCode(404);  
printStatusCode('404');
```

```
// error: Property 'toUpperCase' does not exist on type 'string | number'.  
function printStatusCode(code: string | number) {  
  console.log(`My status code is ${code.toUpperCase()}.`)  
}
```



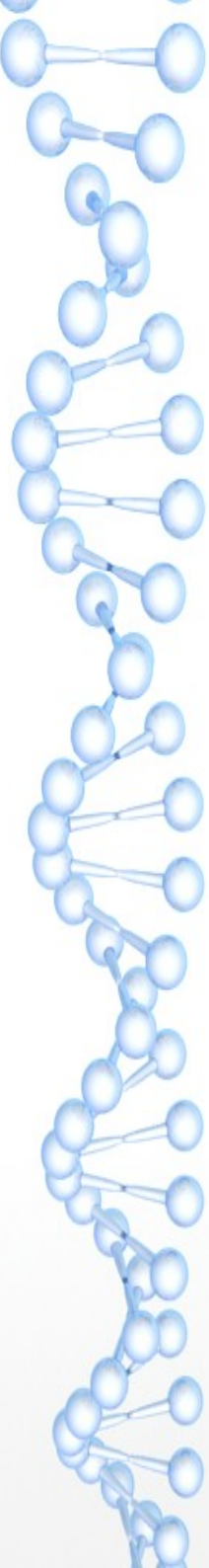
Outline

- Intro to TypeScript
- TypeScript Data Types
- **Function**
- Casting
- Class
- Generics



Functions

- Functions
- Type Annotation for parameter and return type
- Optional and Default Parameter
- Function Overloads
- Fat Arrow functions
- Rest parameters



Functions

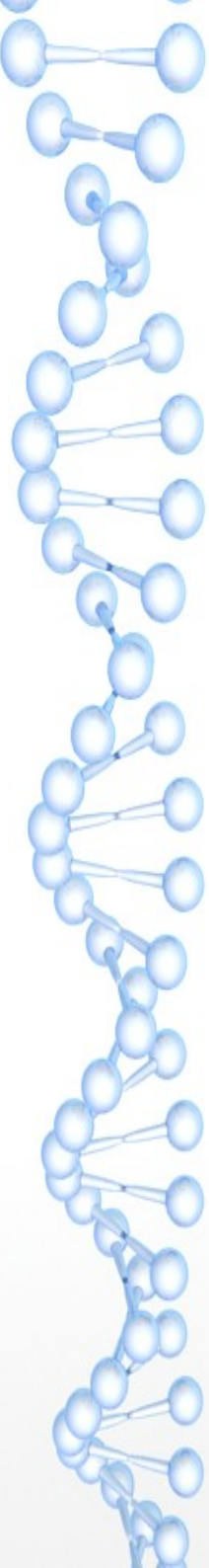
- JavaScript supports two types of functions
 - Named function
 - Anonymous (i.e., nameless) function

```
// Named function
function add(x, y) {
    return x + y;
}

// Anonymous function
let myAdd = function(x, y) { return x + y; };
```

- TypeScript functions can similarly be named or anonymous
- Just as in JavaScript, TypeScript functions can refer to variables outside of the function body. When they do so, they are said to capture these variables

```
let z = 100;
function addToZ(x, y) {
    return x + y + z;
}
```

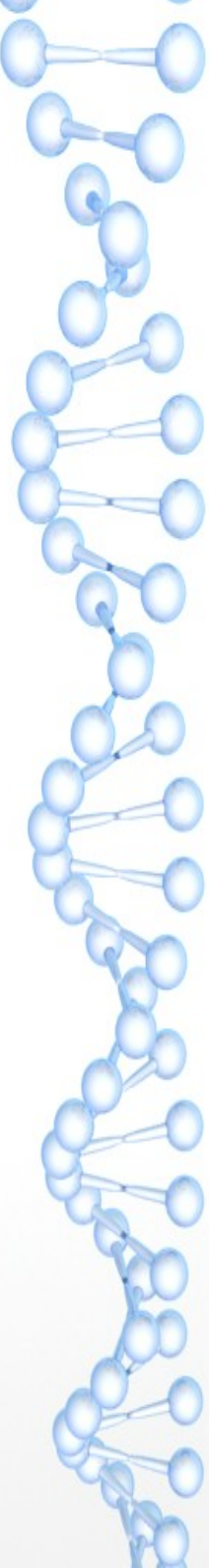


Typing the Function

- We can add types to each of the parameters and then to the function itself to add a return type
- TypeScript can figure the return type out by looking at the return statements, so we can also optionally leave this off in many cases

```
// Named function
function add(x: number, y: number): number {
    return x + y;
}

//Anonymous function
let myAdd = function(x: number, y: number): number { return x + y; };
```



Arrow Functions

- Fat arrow notation
 - Used for anonymous functions
 - i.e. for function expressions or lambda functions in other languages.
- Syntax
 - (param1, param2, ..., paramN) => expression
- Using fat arrow =>
 - function keyword dropped
 - Parameters are passed in the parenthesis (), and the function expression is enclosed within the curly brackets {}.

Arrow Functions Example

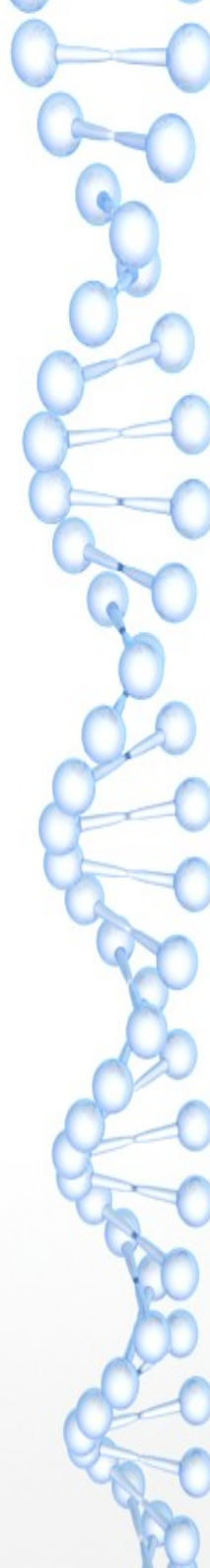
Arrow function

```
let sum = (x: number, y: number): number => {  
    return x + y;  
}  
sum(10, 20); //returns 30
```

- sum is an arrow function
- (x:number, y:number) denotes the parameter types
- :number specifies the return type
- The fat arrow => separates the function parameters and the function body
- The right side of => can contain one or more code statements.

Corresponding JavaScript code

```
var sum = function (x, y) {  
    return x + y;  
}
```

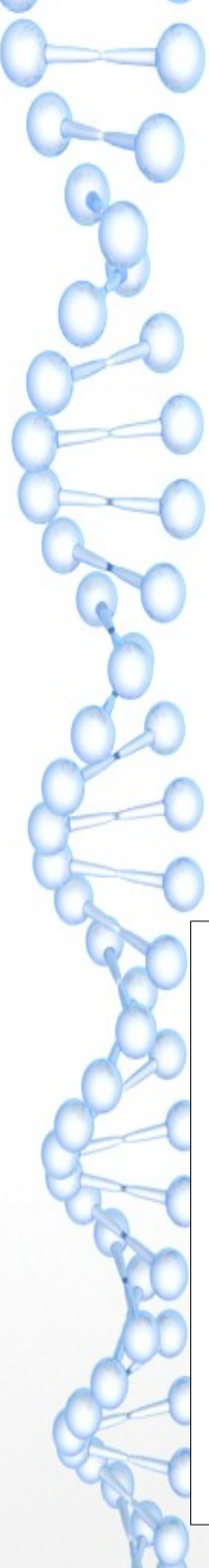


Optional and Default Parameters

- In TypeScript

- Every parameter is assumed to be required by the function
- Compiler also assumes that these parameters are the only parameters that will be passed to the function

```
function buildName(firstName: string, lastName: string) {  
    return firstName + " " + lastName;  
}  
let result1 = buildName("Bob");           // error, too few parameters  
let result2 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result3 = buildName("Bob", "Adams");    // just right
```

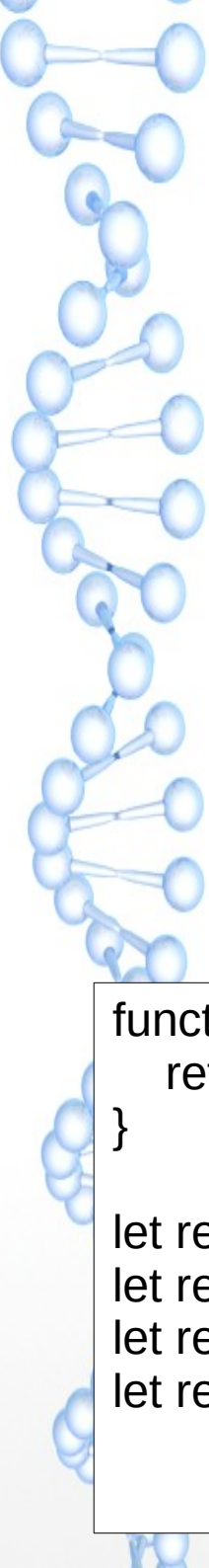


Optional and Default Parameters

- In JavaScript
 - Every parameter is optional, and users may leave them off as they see fit
 - When they do, their value is undefined
 - We can get this functionality in TypeScript by adding a ? to the end of parameters we want to be optional
 - For example, let's say we want the lastName parameter from above to be optional
 - Any optional parameters must follow required parameters
 - Had we wanted to make the first name optional rather than the last name, we would need to change the order of parameters in the function, putting the first name last in the list.

```
function buildName(firstName: string, lastName?: string) {  
    if (lastName)  
        return firstName + " " + lastName;  
    else  
        return firstName;  
}
```

```
let result1 = buildName("Bob");           // works correctly now  
let result2 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result3 = buildName("Bob", "Adams");    // just right
```

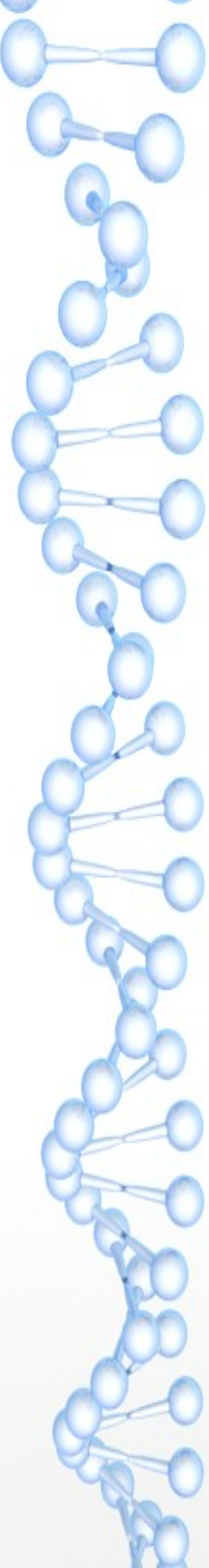


Optional and Default Parameters

- In TypeScript
 - We can also set a value that a parameter will be assigned if the user does not provide one, or if the user passes undefined in its place
 - These are called default-initialized parameters
 - Default the last name to "Smith"

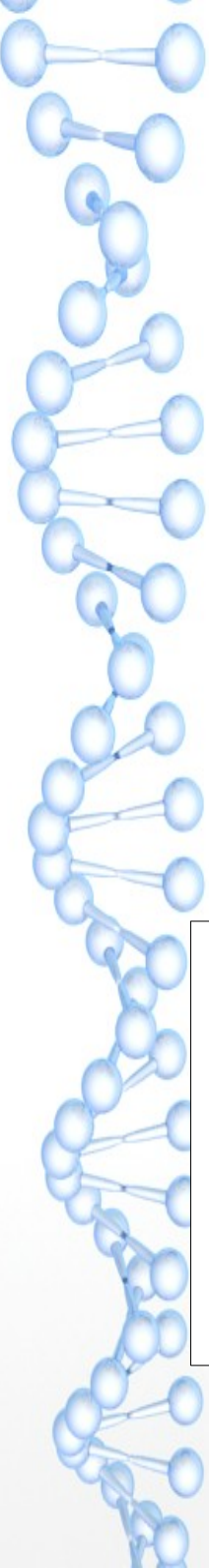
```
function buildName(firstName: string, lastName = "Smith") {  
    return firstName + " " + lastName;  
}
```

```
let result1 = buildName("Bob");           // works correctly now, returns "Bob Smith"  
let result2 = buildName("Bob", undefined); // still works, also returns "Bob Smith"  
let result3 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result4 = buildName("Bob", "Adams");      // ah, just right
```



Optional and Default Parameters

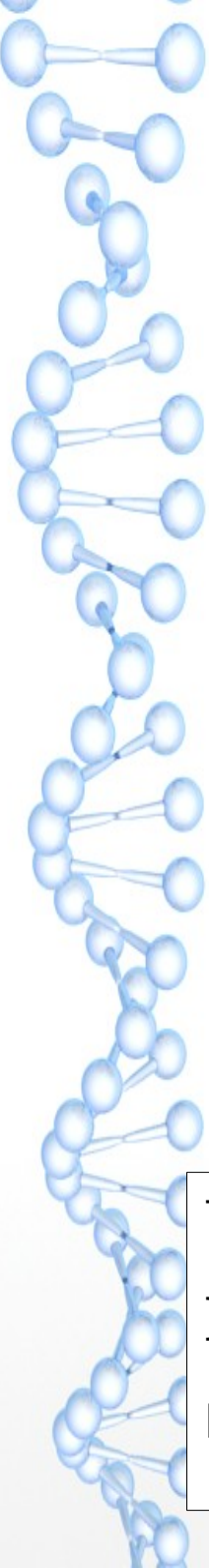
- Default-initialized parameters that come after all required parameters are treated as optional, and just like optional parameters, can be omitted when calling their respective function



Optional and Default Parameters

- Unlike plain optional parameters, default-initialized parameters don't need to occur after required parameters
- If a default-initialized parameter comes before a required parameter, users need to explicitly pass undefined to get the default initialized value

```
function buildName(firstName = "Will", lastName: string) {  
    return firstName + " " + lastName;  
}  
  
let result1 = buildName("Bob");           // error, too few parameters  
let result2 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result3 = buildName("Bob", "Adams");    // okay and returns "Bob Adams"  
let result4 = buildName(undefined, "Adams"); // okay and returns "Will Adams"
```

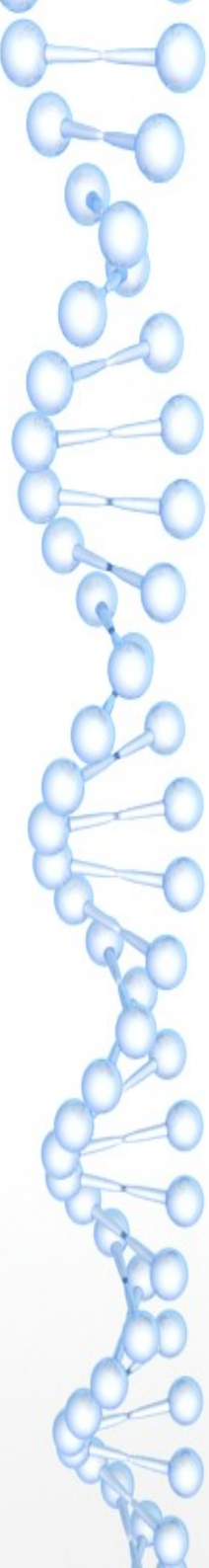


Rest Parameters

- Required, optional, and default parameters all have one thing in common
 - They talk about one parameter at a time
 - Sometimes, you want to work with multiple parameters as a group, or you may not know how many parameters a function will ultimately take
 - In JavaScript, you can work with the arguments directly using the arguments variable that is visible inside every function body.
 - In TypeScript, you can gather these arguments together into a variable
 -
 - Rest parameters are treated as a boundless number of optional parameters
 - When passing arguments for a rest parameter, you can use as many as you want; you can even pass none
 - The compiler will build an array of the arguments passed in with the name given after the ellipsis (...), allowing you to use it in your function

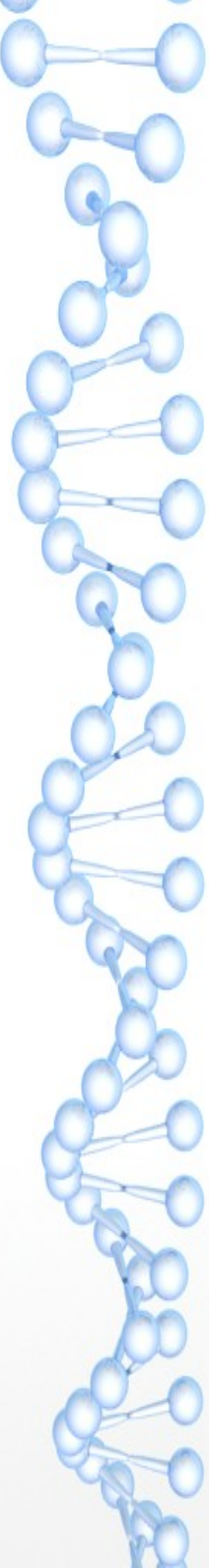
```
function buildName(firstName: string, ...restOfName: string[]) {  
    return firstName + " " + restOfName.join(" ");  
}
```

```
let employeeName = buildName("Joseph", "Samuel", "Lucas", "MacKinzie");
```



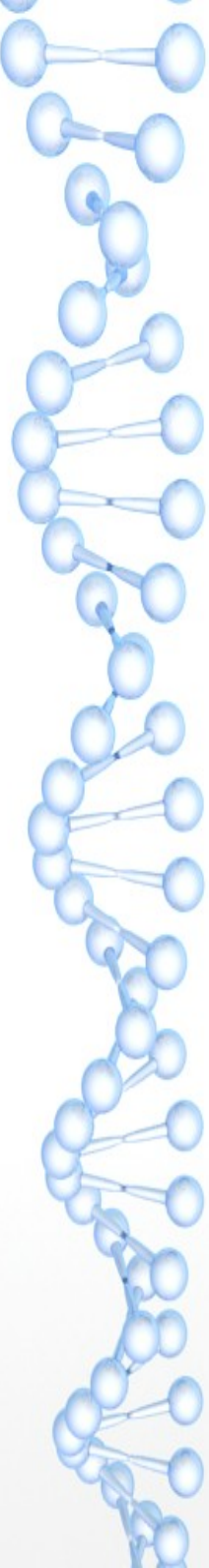
Outline

- Intro to TypeScript
- TypeScript Data Types
- Function
- **Casting**
- Class
- Generics



Casting

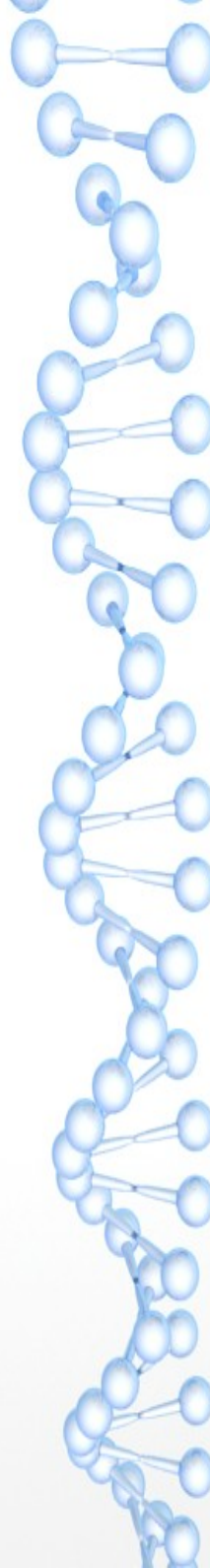
- Sometimes it is necessary to override the type of a variable, e.g., when incorrect types are provided by a library
- Casting is the process of overriding a type



Casting with **as** keyword

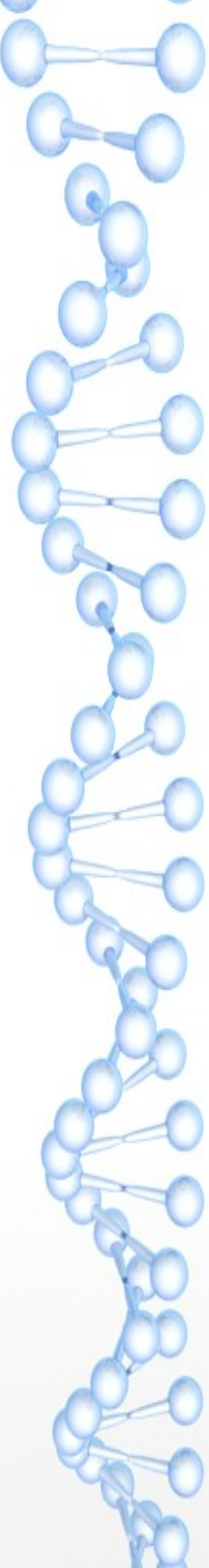
```
//this works  
let x: unknown = 'hello';  
console.log((x as string).length);
```

```
//this does not work since numbers don't have a length property  
let x: unknown = 4;  
console.log((x as string).length);
```



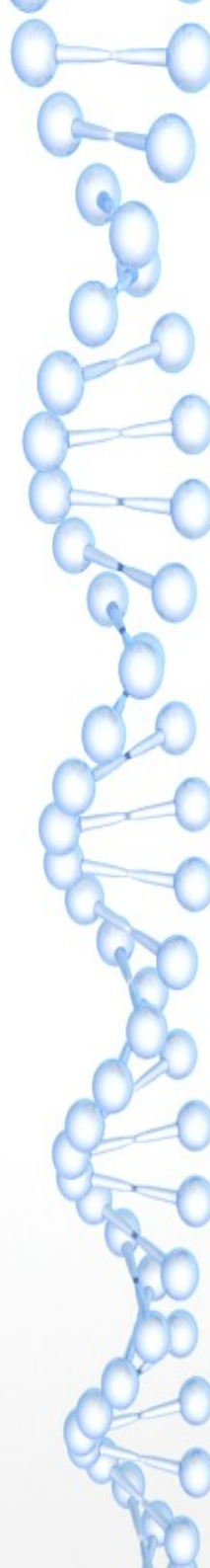
Casting with `<>`

```
let x: unknown = 'hello';  
console.log((<string>x).length);
```



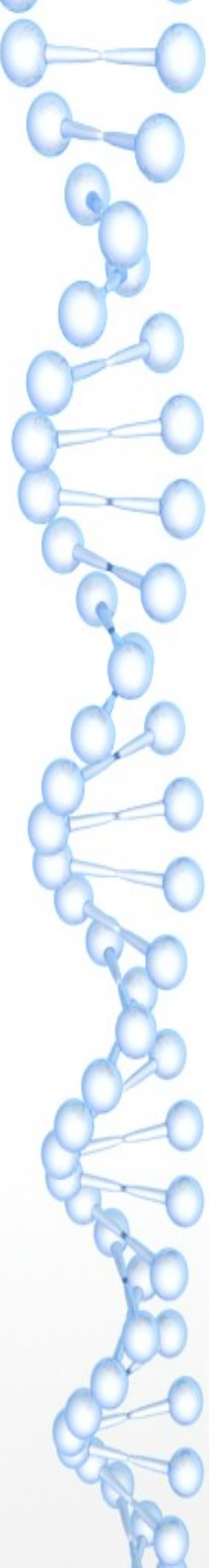
Outline

- Intro to TypeScript
- TypeScript Data Types
- Function
- Casting
- **Class**
- Generics



Class

- Uses
 - Properties and fields to store data
 - Methods to define behaviour
- Inheritance
 - Typescript supports inheritance of class through
 - extends keyword
 - super keyword
- Class
 - Can implement interfaces



Classes

- TypeScript supports class-based object-oriented programming
- Here we're going to create a Student class with a constructor and a few public fields. Notice that classes and interfaces play well together, letting the programmer decide on the right level of abstraction
- Also of note, the use of **public** on arguments to the constructor is a shorthand that allows us to automatically create properties with that name

greeter.ts

```
class Student {
  fullName: string;
  constructor(public firstName: string, public middleInitial: string, public lastName: string) {
    this.fullName = firstName + " " + middleInitial + " " + lastName;
  }
}

interface Person {
  firstName: string;
  lastName: string;
}

function greeter(person : Person) {
  return "Hello, " + person.firstName + " " + person.lastName;
}

let user = new Student("Jane", "M.", "User");
document.body.innerHTML = greeter(user);
```

greeter.js

```
var Student = /** @class */ (function () {
  function Student(firstName, middleInitial, lastName) {
    this.firstName = firstName;
    this.middleInitial = middleInitial;
    this.lastName = lastName;
    this.fullName = firstName + " " + middleInitial + " " + lastName;
  }
  return Student;
}());

function greeter(person) {
  return "Hello, " + person.firstName + " " + person.lastName;
}

var user = new Student("Jane", "M.", "User");
document.body.innerHTML = greeter(user);
```

Classes – Example 2

```
class Greeter {  
  greeting: string;  
  constructor(message: string) {  
    this.greeting = message;  
  }  
  greet() {  
    return "Hello, " + this.greeting;  
  }  
}  
let greeter = new Greeter("world");
```

- Class name: Greeter
- 3 members
 - greeting (property)
 - constructor
 - greet (method)
- Keyword **this**
 - Used to refer to one of the members of the class
- Last line
 - Constructs an instance (i.e., object) of the Greeter class using keyword **new**
This calls into the constructor we defined earlier, creating a new object with the Greeter shape, and running the constructor to initialize it



Inheritance

```
class Animal {  
  move(distanceInMeters: number = 0) {  
    console.log(`Animal moved ${distanceInMeters}m.`);  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log('Woof! Woof!');  
  }  
}  
  
const dog = new Dog();  
dog.bark();  
dog.move(10);  
dog.bark();
```

Output

```
Woof! Woof!  
Animal moved 10m.  
Woof! Woof!
```

- Classes inherit properties and methods from base classes
 - Dog is a derived class (subclass) that derives from the Animal base class (super class) using the **extends** keyword
 - Because Dog extends the functionality from Animal, we were able to create an instance of Dog that could both bark() and move()

Inheritance 2

```
class Animal {
  name: string;
  constructor(theName: string) { this.name = theName; }
  move(distanceInMeters: number = 0) {
    console.log(`${this.name} moved ${distanceInMeters}m.`);
  }
}

class Snake extends Animal {
  constructor(name: string) { super(name); }
  move(distanceInMeters = 5) {
    console.log("Slithering...");
    super.move(distanceInMeters);
  }
}

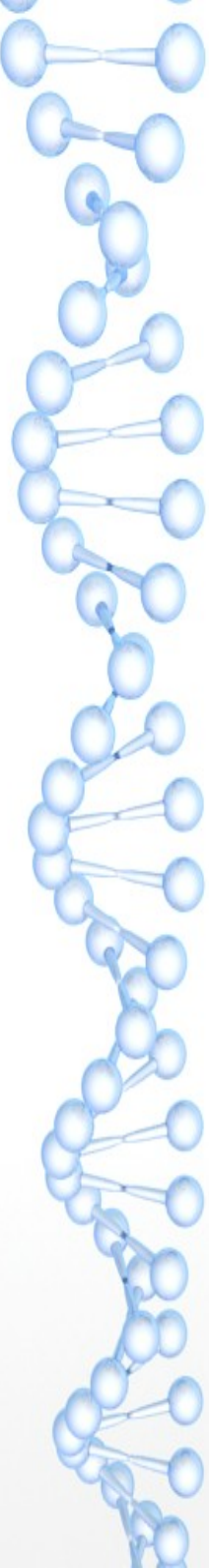
class Horse extends Animal {
  constructor(name: string) { super(name); }
  move(distanceInMeters = 45) {
    console.log("Galloping...");
    super.move(distanceInMeters);
  }
}

let sam = new Snake("Sammy the Python");
let tom: Animal = new Horse("Tommy the Palomino");
sam.move();
tom.move(34);
```

Output

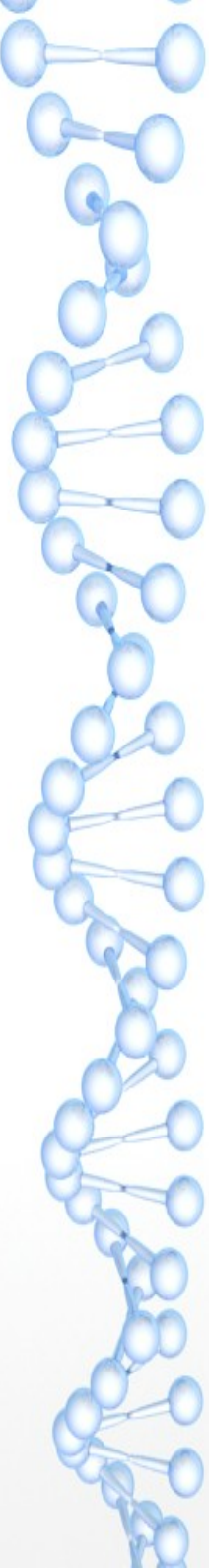
```
Slithering...
Sammy the Python moved 5m.
Galloping...
Tommy the Palomino moved 34m.
```

- Two new subclasses of Animal: Horse and Snake
- Notes
 - Each derived class that contains a constructor function must call `super()` which will execute the constructor of the base class
 - The example also shows how to override methods in the base class with methods that are specialized for the subclass
 - Both Snake and Horse create a move method that overrides the move from Animal, giving it functionality specific to each class
 - Note that even though tom is declared as an Animal, since its value is a Horse, calling `tom.move(34)` will call the overriding method in Horse



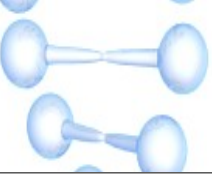
Public, private, and protected modifiers

- Same as in Java
 - Default modifier in Typescript is public, and can be omitted
- TypeScript also supports static property the way Java does
- TypeScript also supports the readonly modifier – see documentation



Abstract Classes

- Similar to Java abstract classes



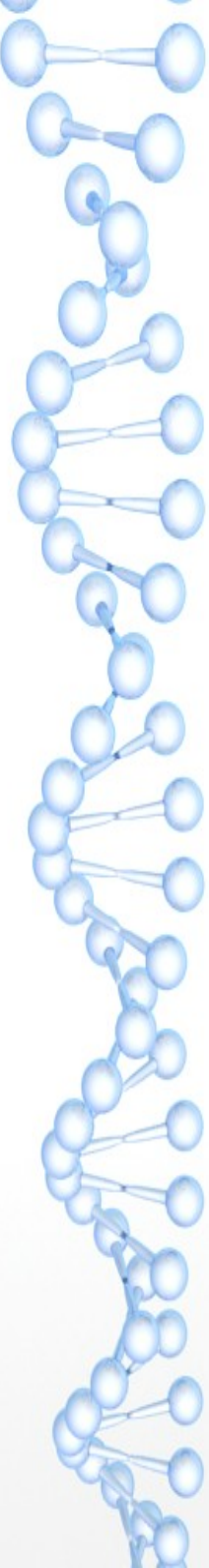
Abstract Classes

```
abstract class Department {  
  constructor(public name: string) { }  
  printName(): void {  
    console.log("Department name: " + this.name);  
  }  
  abstract printMeeting(): void; // must be implemented in derived classes  
}  
class AccountingDepartment extends Department {  
  constructor() {  
    super("Accounting and Auditing"); // constructors in derived classes must call super()  
  }  
  printMeeting(): void {  
    console.log("The Accounting Department meets each Monday at 10am.");  
  }  
  generateReports(): void {  
    console.log("Generating accounting reports...");  
  }  
}  
let department: Department; // ok to create a reference to an abstract type  
department = new Department(); // error: cannot create an instance of an abstract class  
department = new AccountingDepartment(); // ok to create and assign a non-abstract subclass  
department.printName();  
department.printMeeting();  
department.generateReports();
```

Output

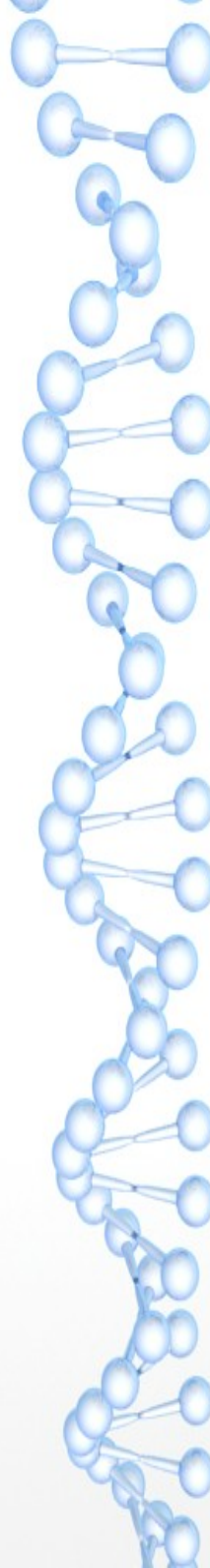
```
Department name: Accounting and Auditing  
The Accounting Department meets each Monday at 10am.  
Generating accounting reports...
```





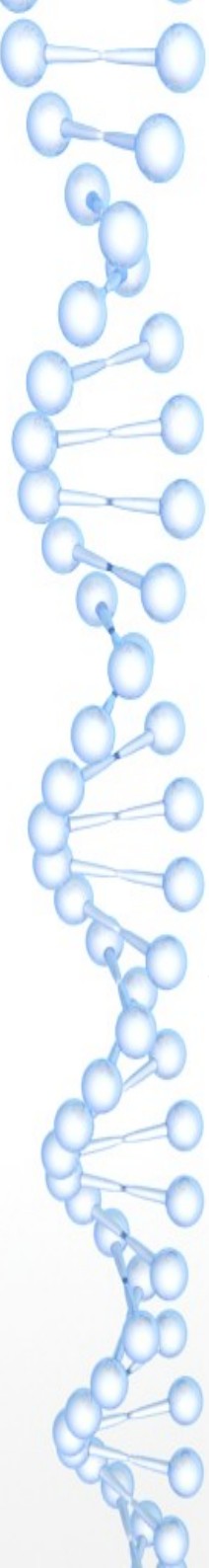
Outline

- Intro to TypeScript
- TypeScript Data Types
- Function
- Casting
- Class
- **Generics**



Generics

- Enable us create reusable components
 - i.e., components that can work over a variety of types rather than just a single one
- Simple example
 - The identity function that will return back whatever is passed in
 - Consider a version that does not use generics and a version that does use generics



Identity Function (Without Generics)

- We either have to give the function a specific type

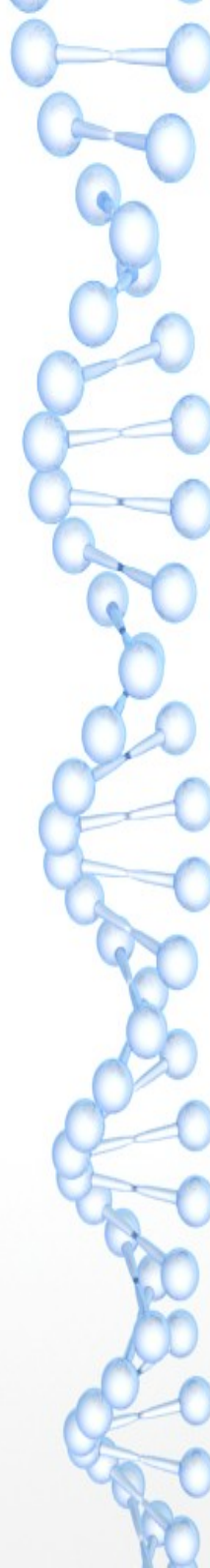
```
function identity(arg: number): number {  
    return arg;  
}
```

- Or describe the identity function using the **any** type

```
function identity(arg: any): any {  
    return arg;  
}
```

- Note

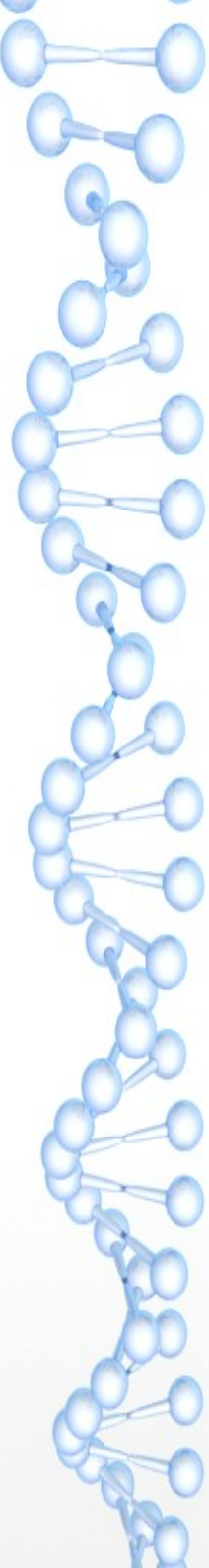
- While using any is certainly generic in that it will cause the function to accept any and all types for the type of arg, we actually are losing the information about what that type was when the function returns
 - If we passed in a number, the only information we have is that any type could be returned.
- We need a way of capturing the type of the argument in such a way that we can also use it to denote what is being returned
- Here, we will use a type variable, a special kind of variable that works on types rather than values



Identity Function with Generics

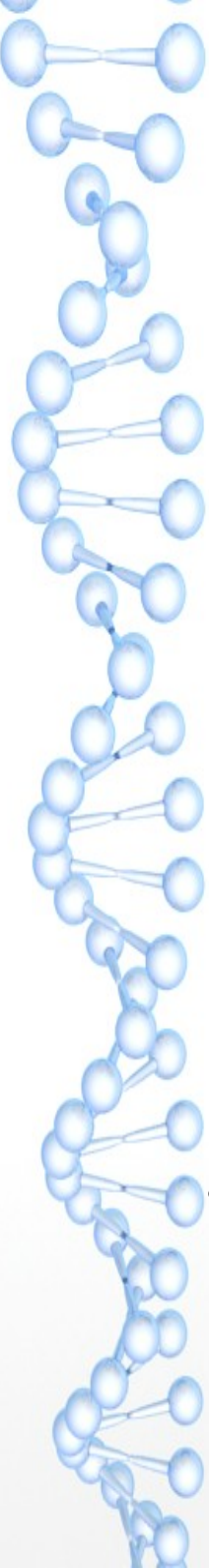
```
function identity<T>(arg: T): T {  
    return arg;  
}
```

- Type variable T has now been added to the identity function
 - T allows us to capture the type the user provides (e.g. number), so that we can use that information later
 - Type T is used for the argument
 - In this example, T also acts as the return type
- We say that this version of the identity function is generic, as it works over a range of types



Identity Function with Generics

- Once we've written the generic identity function, we can call it in one of two ways
 1. Pass all of the arguments, including the type argument, to the function
 - `let output = identity<string>("myString");` // type of output will be 'string'
 2. Use type argument inference (this approach is more common) – that is, we want the compiler to set the value of T for us automatically based on the type of the argument we pass in
 - `let output = identity("myString");` // type of output will be 'string'
 - Notice that we didn't have to explicitly pass the type in the angle brackets (<>)
 - The compiler just looked at the value "myString", and set T to its type
 - Notes
 - Type argument inference can be a helpful tool to keep code shorter and more readable
 - You may need though, to explicitly pass in the type arguments as we did in the previous example when the compiler fails to infer the type, as may happen in more complex examples



Working with Generic Variables

- Let's take another look at our identity function

```
function identity<T>(arg: T): T {  
    return arg;  
}
```

- What if we want to also log the length of the argument `arg` to the console with each call?
 - We might be tempted to write this:

```
function loggingIdentity<T>(arg: T): T {  
    console.log(arg.length); // Error: T doesn't have .length  
    return arg;  
}
```

- Remember, type variables stand in for any and all types, so someone using this function could have passed in a number instead, which does not have a `.length` member

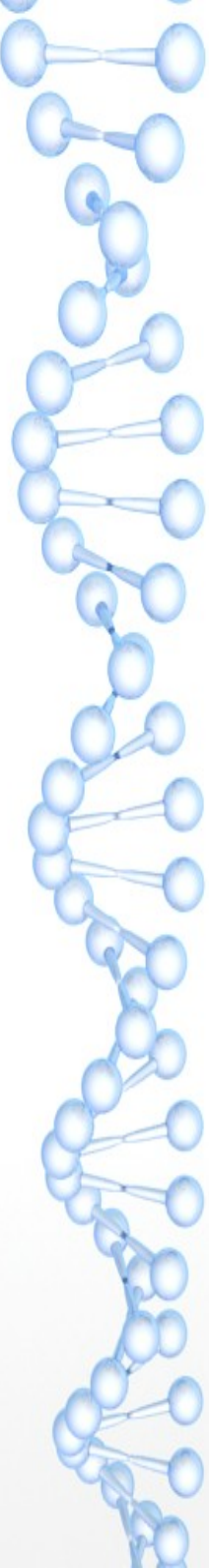
Working with Generic Variables

- Assume we intended this function to work on arrays of T rather than T directly
- Since we're working with arrays, the .length member should be available
- We can describe this just like we would create arrays of other types

```
function loggingIdentity<T>(arg: T[]): T[] {  
    console.log(arg.length); // Array has a .length, so no more error  
    return arg;  
}
```

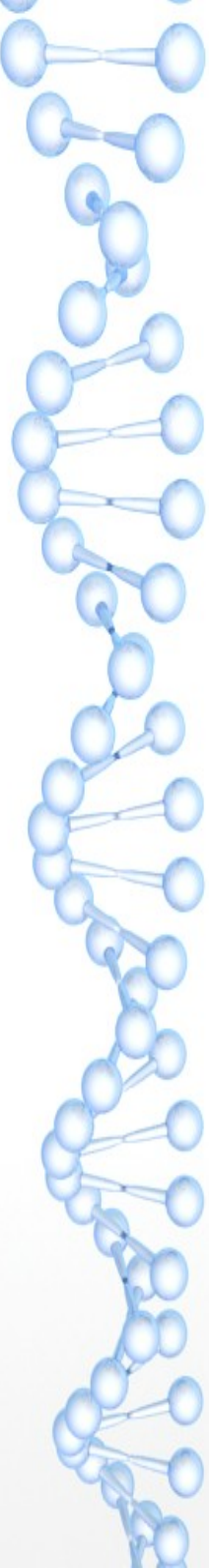
- The generic function loggingIdentity takes a type parameter T, and an argument arg which is an array of Ts, and returns an array of Ts
 - If we passed in an array of numbers, we'd get an array of numbers back out, as T would bind to number
 - This allows us to use our generic type variable T as part of the types we're working with, rather than the whole type, giving us greater flexibility
- We can alternatively write the sample example this way

```
function loggingIdentity<T>(arg: Array<T>): Array<T> {  
    console.log(arg.length); // Array has a .length, so no more error  
    return arg;  
}
```



Generic Types

- Previous sections
 - We created generic identity functions that worked over a range of types
- In this section
 - We'll explore the type of the functions themselves and how to create generic interfaces



Generic Types

```
function identity<T>(arg: T): T {  
    return arg;  
}
```

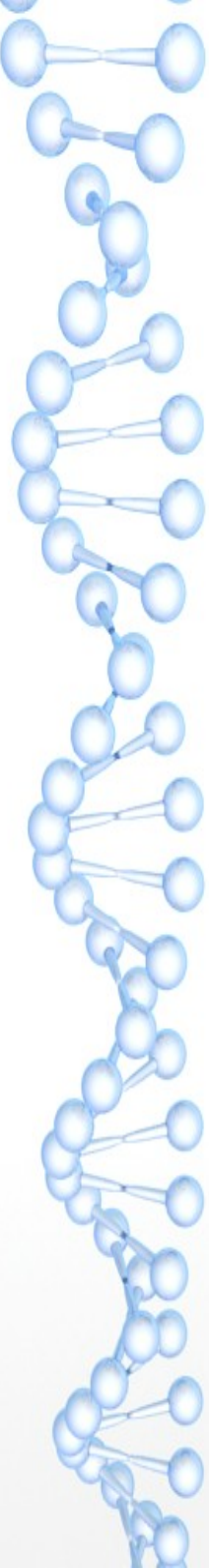
- The type of generic functions is just like those of non-generic functions, with the type parameters listed first, similarly to function declarations
 - let myIdentity: <T>(arg: T) => T = identity;
- Equivalently
 - let myIdentity: <U>(arg: U) => U = identity;
- We can also write the generic type as a call signature of an object literal type
 - let myIdentity: {<T>(arg: T): T} = identity;
 - This leads us to writing our first generic interface
 - Move the object literal to an interface

Generic Interface

```
interface GenericIdentityFn {  
    <T>(arg: T): T;  
}  
function identity<T>(arg: T): T {  
    return arg;  
}  
let myIdentity: GenericIdentityFn = identity;
```

- We may want to move the generic parameter to be a parameter of the whole interface
 - This lets us see what type(s) we're generic over (e.g. Dictionary<string> rather than just Dictionary)
 - i.e., type parameter now visible to all the other members of the interface

```
interface GenericIdentityFn<T> {  
    (arg: T): T;  
}  
function identity<T>(arg: T): T {  
    return arg;  
}  
let myIdentity: GenericIdentityFn<number> = identity;
```

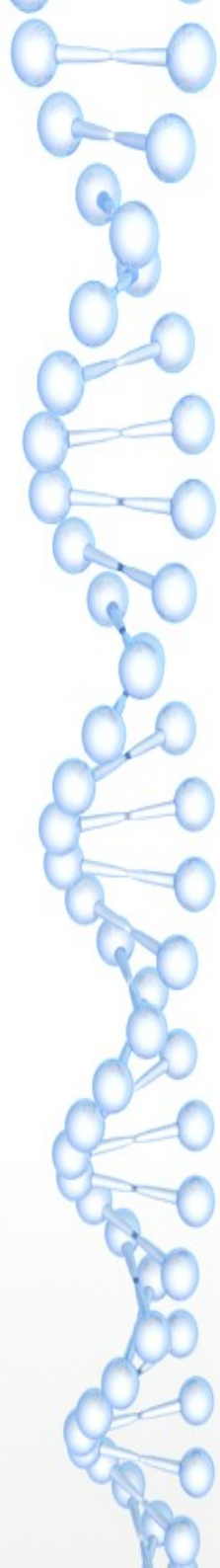


Generic Class

- Has a similar shape to a generic interface
- Generic classes have a generic type parameter list in angle brackets (<>) following the name of the class

```
class GenericNumber<T> {  
    zeroValue: T;  
    add: (x: T, y: T) => T;  
}
```

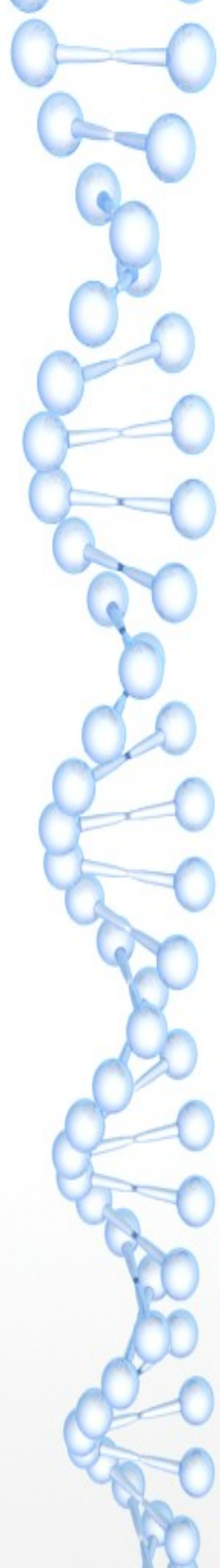
```
let myGenericNumber = new GenericNumber<number>();  
myGenericNumber.zeroValue = 0;  
myGenericNumber.add = function(x, y) { return x + y; };
```



Generic Class

- We could have instead used string or even more complex objects, instead of number

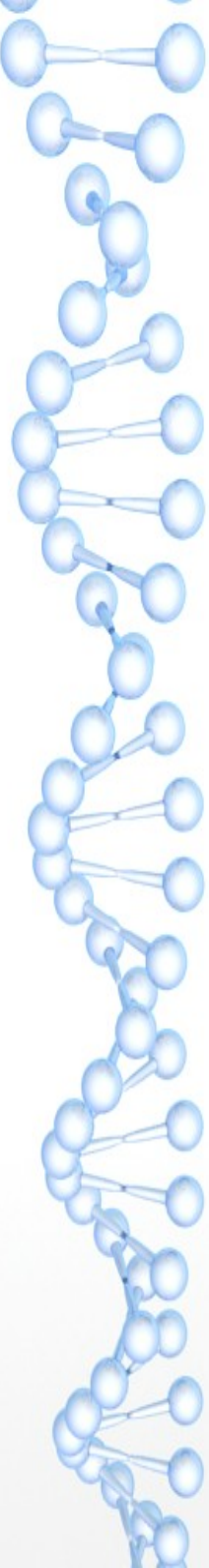
```
class GenericNumber<T> {  
  zeroValue: T;  
  add: (x: T, y: T) => T;  
}  
  
let stringNumeric = new GenericNumber<string>();  
stringNumeric.zeroValue = "";  
stringNumeric.add = function(x, y) { return x + y; };  
  
alert(stringNumeric.add(stringNumeric.zeroValue, "test"));
```



Iterators

- Iterables

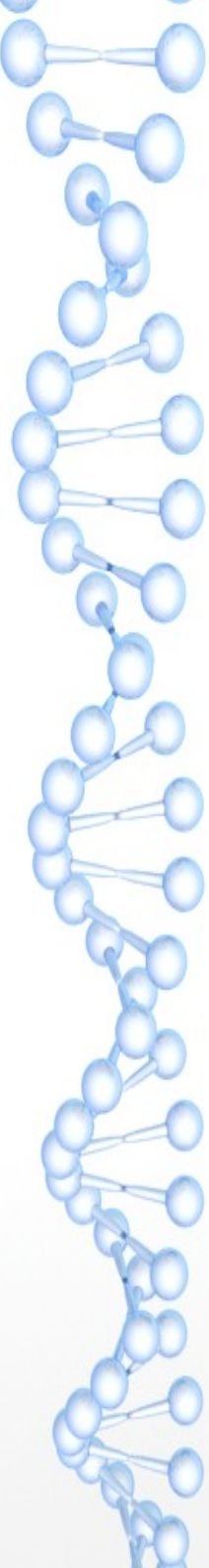
- An object is deemed iterable if it has an implementation for the `Symbol.iterator` property (See symbol type in documentation)
- Some built-in types like `Array`, `Map`, `Set`, `String`, `Int32Array`, `Uint32Array`, etc. have their `Symbol.iterator` property already implemented
- `Symbol.iterator` function on an object is responsible for returning the list of values to iterate on



for..of statements

- for..of loops over an iterable object, invoking the Symbol.iterator property on the object

```
let someArray = [1, "string", false];  
  
for (let entry of someArray) {  
    console.log(entry); // 1, "string", false  
}
```



for..of vs. for..in statements

- Both for..of and for..in statements iterate over lists
 - The values iterated on are different though
 - for..in returns a list of keys on the object being iterated
 - for..of returns a list of values of the numeric properties of the object being iterated

```
let list = [4, 5, 6];  
  
for (let i in list) {  
    console.log(i); // "0", "1", "2",  
}  
  
for (let i of list) {  
    console.log(i); // "4", "5", "6"  
}
```